

LAPORAN PRAKTIKUM

Aplikasi Berbasis Platform

MODUL 4-5

MOBILE



Disusun oleh:

SINTA SARWO

2311102132

S1 IF-11-REG04

Dosen Pengampu:

Cahyo Prihantoro, S.Kom., M.Eng

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

MODUL 4 – Antarmuka Pengguna

DASAR TEORI

1. Antarmuka Pengguna

Antarmuka pengguna (UI) pada Flutter dibangun dengan memecah tampilan menjadi komponen-komponen kecil yang mandiri.

1.1. Pengenalan Widget

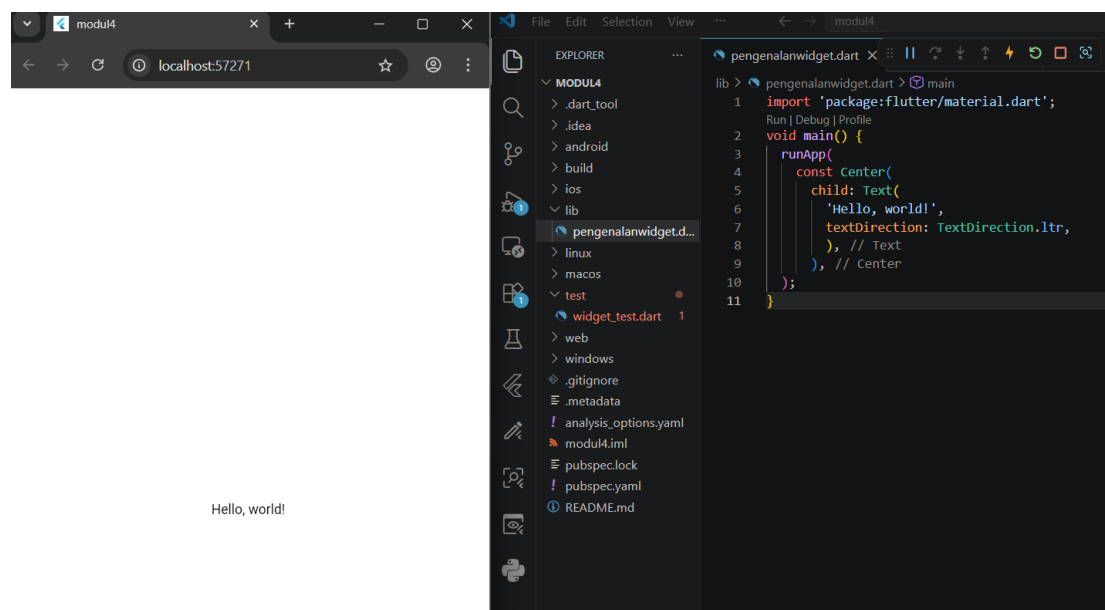
Pengenalan Widget Widget adalah unit dasar pembangun aplikasi Flutter yang terinspirasi dari Reactjs. Setiap widget memiliki state dan konfigurasi sendiri, serta akan melakukan rebuild (membuat ulang dirinya) secara otomatis jika terjadi perubahan data atau status untuk memastikan tampilan selalu sinkron.

Source code:

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    const Center(
      child: Text(
        'Hello, world!',
        textDirection: TextDirection.ltr,
      ),
    ),
  );
}
```

Screenshot:



1.2. Container

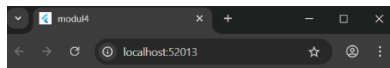
Container Widget serbaguna yang berfungsi sebagai wadah untuk membungkus widget lain dan memberikan elemen visual seperti kotak. Container mendukung dekorasi (BoxDecoration) seperti warna latar, border, atau bayangan, serta pengaturan jarak melalui margin dan padding.

Source Code:

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    Center(
      child: Container(
        margin: const EdgeInsets.all(10.0),
        color: Colors.blueGrey[600],
        width: 48.0,
        height: 48.0,
      ),
    ),
  );
}
```

Screenshot:



Penjelasan:

Widget container ini yang berfungsi untuk membuat kotak, menetapkan ukuran (lebar dan tinggi), memberikan warna latar, serta mengatur jarak luar (margin) dari elemen di sekitarnya.



1.3. GridView

GridView Widget yang menyusun elemen dalam format baris dan kolom, serupa dengan Array 2D. GridView ideal digunakan untuk menampilkan kumpulan item seperti galeri gambar, teks, atau ikon dalam tata letak kisi.

Source code:

```
import 'package:flutter/material.dart';

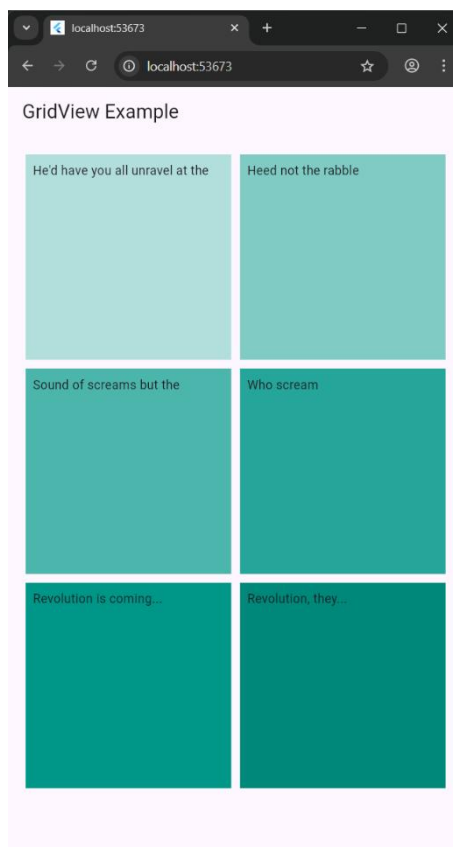
void main() {
  runApp(
    MaterialApp(
      debugShowCheckedModeBanner: false, // Menghilangkan banner debug
      home: Scaffold(
        appBar: AppBar(
          title: const Text('GridView Example'),
        ),
        body: GridView.count(
          primary: false,
          padding: const EdgeInsets.all(20),
          crossAxisSpacing: 10,
          mainAxisSpacing: 10,
          crossAxisCount: 2,
          children: <Widget>[
            Container(
              padding: const EdgeInsets.all(8),
              color: Colors.teal[100],
              child: const Text("He'd have you all unravel at
the"),
            ),
            Container(
              padding: const EdgeInsets.all(8),
              color: Colors.teal[200],
              child: const Text('Heed not the rabble'),
            ),
            Container(
              padding: const EdgeInsets.all(8),
              color: Colors.teal[300],
              child: const Text('Sound of screams but the'),
            ),
          ],
        ),
      ),
    ),
  );
}
```

```

        Container(
          padding: const EdgeInsets.all(8),
          color: Colors.teal[400],
          child: const Text('Who scream'),
        ),
        Container(
          padding: const EdgeInsets.all(8),
          color: Colors.teal[500],
          child: const Text('Revolution is coming...'),
        ),
        Container(
          padding: const EdgeInsets.all(8),
          color: Colors.teal[600],
          child: const Text('Revolution, they...'),
        ),
      ],
    ),
  ),
);
}

```

Screenshot:



Penjelasan:

Pada bagian konten utamanya, `GridView.count` digunakan untuk menyusun elemen-elemen ke dalam tata letak berbentuk dua kolom kotak-kotak yang dapat digeser. Masing-masing kotak tersebut dibentuk menggunakan widget `Container` yang berfungsi untuk memberikan warna latar dan jarak ke dalam (`padding`), di mana setiap kotaknya membungkus widget `Text` untuk menampilkan potongan teks di layarnya.

1.4. ListView

ListView Widget scroll yang paling umum digunakan untuk menampilkan daftar komponen secara berurutan. Secara default, ListView menggunakan parameter children untuk menampung daftar widget yang ingin ditampilkan.

1.4.1. ListView.builder

Digunakan untuk menangani daftar data yang besar atau dinamis secara efisien. Widget ini hanya merender elemen yang terlihat di layar menggunakan parameter itemBuilder dan itemCount.

Source code:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    final List<String> entries = <String>['A', 'B', 'C'];
    final List<int> colorCodes = <int>[600, 500, 100];

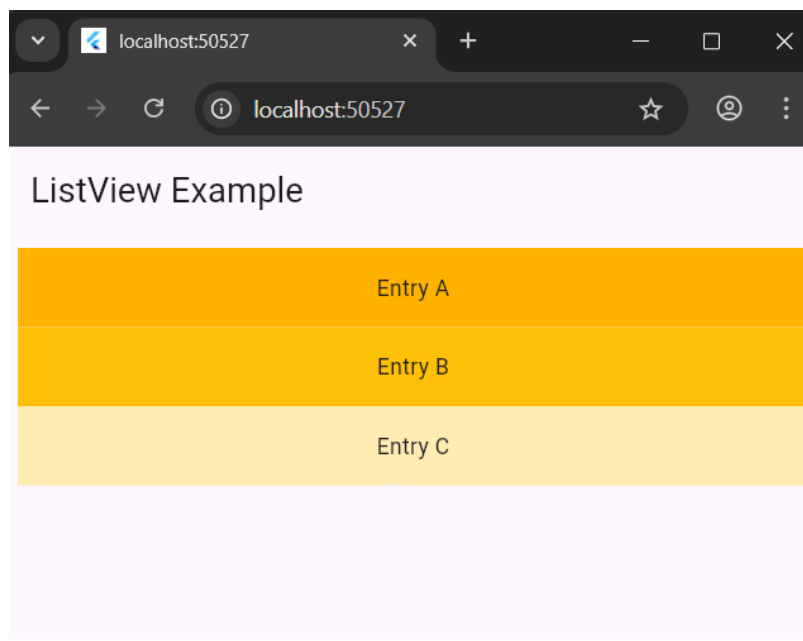
    return MaterialApp(
      debugShowCheckedModeBanner: false, // Opsional:
      menyembunyikan pita debug
      home: Scaffold(
        appBar: AppBar(
          title: const Text('ListView Example'),
        ),
        body: ListView.builder(
          padding: const EdgeInsets.all(8),
          itemCount: entries.length,
          itemBuilder: (BuildContext context, int index) {
            return Container(
              height: 50,
              color: Colors.amber[colorCodes[index]],
              child: Center(child: Text('Entry
${entries[index]}')),
            );
          },
        ),
      ),
    );
  }
}
```

```

        );
    }
    ),
    ),
);
}
}

```

Screenshot:



Penjelasan:

Pada bagian utamanya, kode ini memanfaatkan `ListView.builder` untuk secara dinamis membangun sebuah daftar yang dapat digeser berdasarkan jumlah data yang ada di dalam variabel array `entries`. Untuk setiap baris datanya, `ListView.builder` memanggil widget `Container` yang berfungsi mengatur tinggi dan warna latar secara berurutan mengambil dari array `colorCodes`, sementara widget `Center` dan `Text` digunakan untuk memastikan data tulisan dari array tersebut tampil tepat di tengah-tengah masing-masing kotak.

1.4.2. ListView.separated

Memiliki fungsi yang mirip dengan builder, namun memiliki parameter tambahan wajib yaitu `separatorBuilder` untuk menyisipkan widget pemisah (seperti garis pembagi) di antara setiap item.

Source code:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

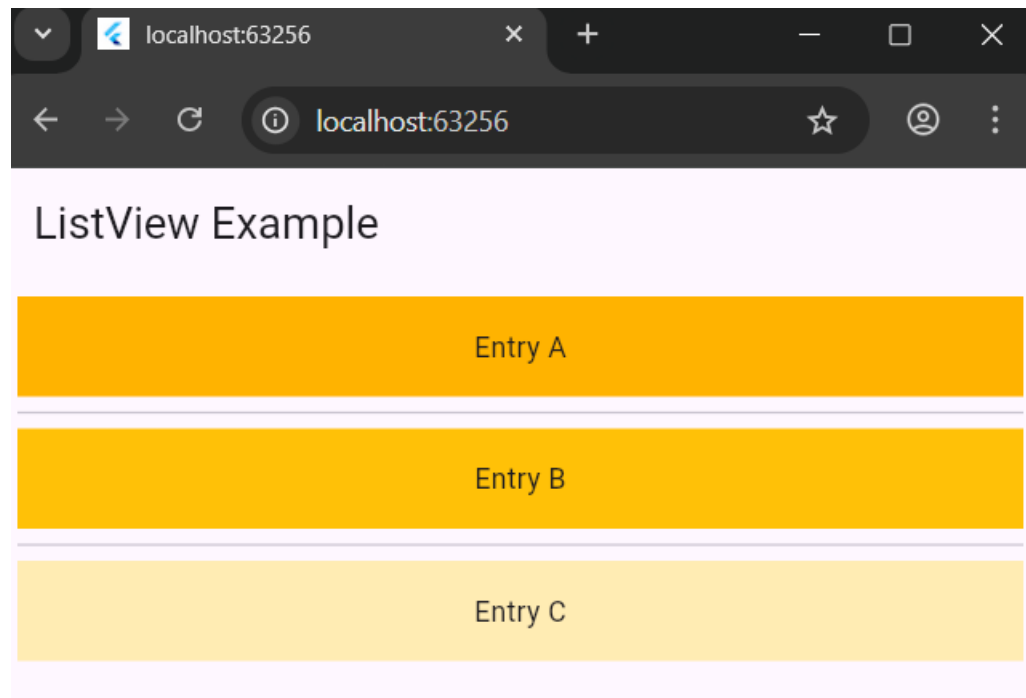
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    final List<String> entries = <String>['A', 'B', 'C'];
    final List<int> colorCodes = <int>[600, 500, 100];

    return MaterialApp(
      debugShowCheckedModeBanner: false, // Opsional:
      menyembunyikan pita debug
      home: Scaffold(
        appBar: AppBar(
          title: const Text('ListView Example'),
        ),
        body: ListView.separated(
          padding: const EdgeInsets.all(8),
          itemCount: entries.length,
          itemBuilder: (BuildContext context, int index) {
            return Container(
              height: 50,
              color: Colors.amber[colorCodes[index]],
              child: Center(child: Text('Entry
${entries[index]}')),
            );
          },
          separatorBuilder: (BuildContext context, int index) {
            return const Divider();
          },
        ),
      ),
    );
  }
}
```

```
        },  
        ),  
    ),  
    );  
}  
}
```

Screenshot:



Penjelasan:

Pada bagian utamanya, kode ini memanfaatkan `ListView.separated` untuk secara dinamis membangun daftar data dari sebuah array yang dapat digeser, dengan tambahan fungsi khusus `separatorBuilder` yang memanggil widget `Divider` untuk menyisipkan garis pembatas otomatis di antara setiap baris itemnya. Untuk membangun setiap baris data tersebut, widget `Container` digunakan untuk mengatur tinggi dan warna latar belakang secara berurutan, sementara widget `Center` dan `Text` berfungsi untuk memastikan data teks tampil tepat di tengah-tengah masing-masing kotak.

1.5. Stack

Stack Widget yang memungkinkan penumpukan elemen secara berlapis (tumpang tindih). Stack sangat berguna untuk membuat tampilan overlay, seperti menempatkan teks di atas gambar atau tombol melayang di atas konten lain.

Source code:

stack1.dart

```
import 'package:flutter/material.dart';

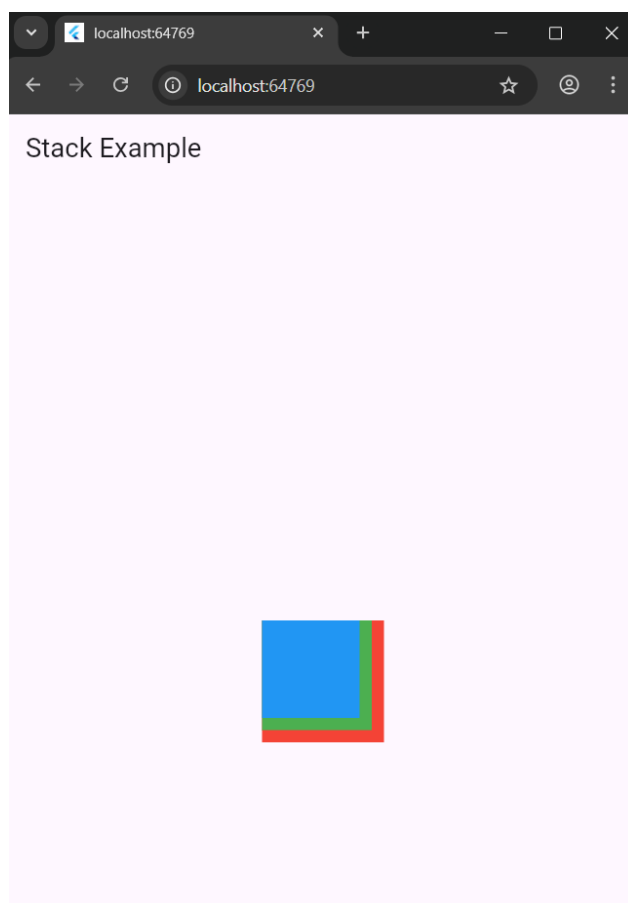
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Stack Example'),
        ),
        body: Center( // Menengahkan posisi Stack di dalam layar
          // Ini adalah kode Stack milikmu
          child: Stack(
            children: <Widget>[
              Container(
                width: 100,
                height: 100,
                color: Colors.red,
              ),
              Container(
                width: 90,
                height: 90,
                color: Colors.green,
              ),
              Container(
                width: 80,
```

```
        height: 80,  
        color: Colors.blue,  
      ),  
    ],  
  ),  
),  
),  
),  
);  
}  
}
```

Screenshot:



Penjelasan:

Dalam Flutter, widget Stack memang berfungsi untuk menumpuk elemen-elemen secara berlapis (layering), di mana widget yang ditulis paling awal di dalam daftar children akan berada di lapisan paling dasar.

MODUL 5 – Antarmuka Pengguna Lanjutan

DASAR TEORI

2. Antarmuka pengguna Lanjutan

2.1. Row

Row Widget merupakan tata letak yang berfungsi untuk menyusun daftar widget secara horizontal (mendatar) dari kiri ke kanan. Untuk menghindari masalah overflow (konten melebihi lebar layar), sering kali digunakan widget Expanded agar widget di dalamnya dapat menyesuaikan dengan ruang yang tersedia.

Source code:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

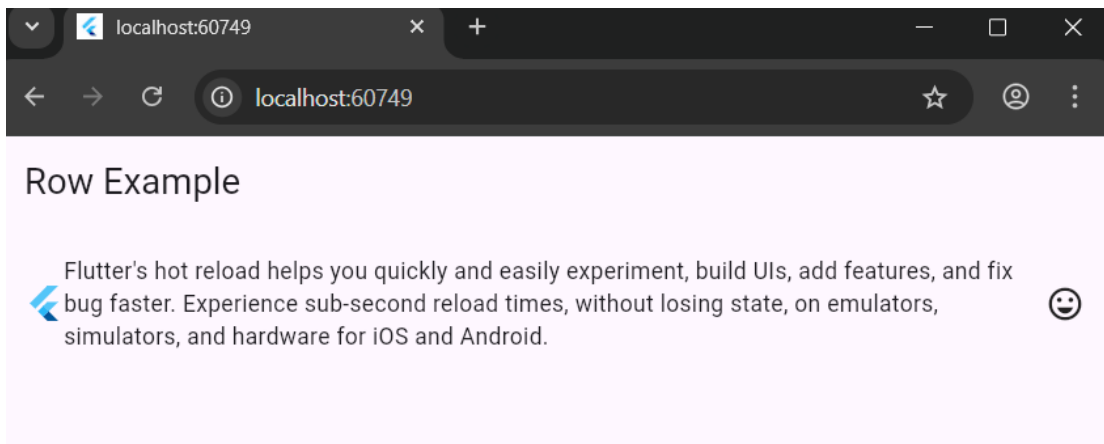
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Row Example'),
        ),
        body: Padding(
          padding: const EdgeInsets.all(16.0),
          child: Row(
            children: <Widget>[
              const FlutterLogo(),
              const Expanded(
                child: Text("Flutter's hot reload helps you quickly
and easily experiment, build UIs, add features, and fix bug faster.
Experience sub-second reload times, without losing state, on
emulators, simulators, and hardware for iOS and Android."),
              ),
              const Icon(Icons.sentiment_very_satisfied),
            ],
          ),
        ),
      ),
    );
  }
}
```

```

        ],
      ),
    ),
  ),
),
);
}
}

```

Screenshot:



Penjelasan:

Pada kode ini, *Row* menyusun widget secara horizontal. Penggunaan widget *Expanded* pada *Text* sangat penting di sini. Tanpa *Expanded*, teks yang panjang akan terus memanjang ke kanan hingga melewati batas layer yang menyebabkan error garis kuning-hitam atau overflow. Dengan *Expanded*, *Text* akan dipaksa untuk mengisi sisa ruang kosong yang ada di antara *FlutterLogo* dan *Icon*, sehingga teksnya akan secara otomatis turun ke baris baru menyesuaikan lebar layar.

2.2. Column

Sedangkan *Column* Widget tata letak yang menyusun daftar widget secara vertikal (tegak lurus) dari atas ke bawah. Kontrol posisi dan perataan elemen dalam *Column* dapat diatur menggunakan properti *crossAxisAlignment* dan *mainAxisSize*.

Source code:

```

import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

```

```

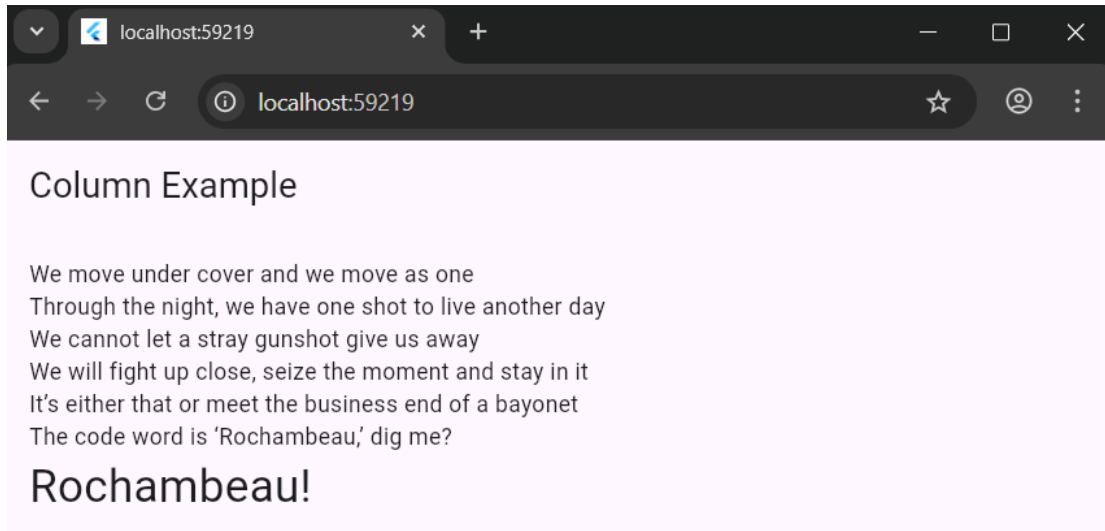
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Column Example'),
        ),
        body: Builder(
          builder: (BuildContext innerContext) {
            return Padding(
              padding: const EdgeInsets.all(16.0),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                mainAxisAlignment: MainAxisAlignment.min,
                children: <Widget>[
                  const Text('We move under cover and we move as
one'),
                  const Text('Through the night, we have one shot
to live another day'),
                  const Text('We cannot let a stray gunshot give us
away'),
                  const Text('We will fight up close, seize the
moment and stay in it'),
                  const Text('It's either that or meet the business
end of a bayonet'),
                  const Text('The code word is 'Rochambeau,' dig
me?'),
                  Text('Rochambeau!', style:
DefaultTextStyle.of(innerContext).style.apply(fontSizeFactor:
2.0)),
                ],
              ),
            );
          },
        ),
      ),
    );
  }
}

```

```
);  
}  
}
```

Screenshot:



Penjelasan:

Berbeda dengan Row yang menyusun elemen secara horizontal, Column menyusun elemen-elemennya secara vertikal. Properti `crossAxisAlignment: CrossAxisAlignment.start` pada kodemu berfungsi untuk meratakan semua teks di dalam kolom agar rata kiri.

2.3. Nested Rows & Columns

Nested Rows & Columns Teknik penggabungan widget Row dan Column di dalam satu sama lain untuk membangun struktur tata letak yang kompleks dan hierarkis. Hal ini dilakukan dengan memasukkan Row atau Column sebagai bagian dari daftar children widget induknya.

Source code:

```
import 'package:flutter/material.dart';  
  
void main() {  
  runApp(const MyApp());  
}  
  
class MyApp extends StatelessWidget {  
  const MyApp({super.key});
```

```

@override
Widget build(BuildContext context) {
  return MaterialApp(
    title: 'Nested Rows & Columns',
    debugShowCheckedModeBanner: false,
    home: Scaffold(
      appBar: AppBar(
        title: const Text('Modul 5.3 - Strawberry Pavlova'),
        backgroundColor: Colors.redAccent,
      ),
      body: const Center(
        child: PavlovaRecipeCard(),
      ),
    ),
  );
}

class PavlovaRecipeCard extends StatelessWidget {
  const PavlovaRecipeCard({super.key});

  @override
  Widget build(BuildContext context) {
    // 1. BAGIAN TEKS (JUDUL & DESKRIPSI)
    const titleText = Padding(
      padding: EdgeInsets.all(8.0),
      child: Text(
        'Strawberry Pavlova',
        style: TextStyle(fontWeight: FontWeight.w800,
letterSpacing: 0.5, fontSize: 24),
      ),
    );

    const subTitle = Padding(
      padding: EdgeInsets.all(8.0),
      child: Text(
        'Pavlova is a meringue-based dessert named after the
Russian ballerina Anna Pavlova. '
        'Pavlova features a crisp crust and soft, light inside,
topped with fruit and whipped cream.',
        textAlign: TextAlign.center,

```

```

        style: TextStyle(fontFamily: 'Roboto', fontSize: 14),
      ),
    );

// 2. BAGIAN RATING (Row di dalam Container)
var stars = Row(
  mainAxisAlignment: MainAxisAlignment.min,
  children: [
    Icon(Icons.star, color: Colors.green[500], size: 20),
    Icon(Icons.star, color: Colors.green[500], size: 20),
    Icon(Icons.star, color: Colors.green[500], size: 20),
    const Icon(Icons.star, color: Colors.black, size: 20),
    const Icon(Icons.star, color: Colors.black, size: 20),
  ],
);

final ratings = Container(
  padding: const EdgeInsets.all(10),
  child: Row(
    mainAxisAlignment: MainAxisAlignment.spaceEvenly,
    children: [
      stars,
      const Text(
        '170 Reviews',
        style: TextStyle(
          color: Colors.black,
          fontWeight: FontWeight.w800,
          fontFamily: 'Roboto',
          letterSpacing: 0.5,
          fontSize: 16,
        ),
      ),
    ],
  ),
);

// 3. ICON LIST (3 Column di dalam 1 Row)
const descTextStyle = TextStyle(
  color: Colors.black,
  fontWeight: FontWeight.w800,
  fontFamily: 'Roboto',

```

```

        letterSpacing: 0.5,
        fontSize: 14,
        height: 2,
    );

    final iconList = DefaultTextStyle.merge(
      style: descTextStyle,
      child: Container(
        padding: const EdgeInsets.all(10),
        child: Row(
          mainAxisAlignment: MainAxisAlignment.spaceEvenly,
          children: [
            Column(
              children: [
                Icon(Icons.kitchen, color: Colors.green[500]),
                const Text('PREP:'),
                const Text('25 min'),
              ],
            ),
            Column(
              children: [
                Icon(Icons.timer, color: Colors.green[500]),
                const Text('COOK:'),
                const Text('1 hr'),
              ],
            ),
            Column(
              children: [
                Icon(Icons.restaurant, color: Colors.green[500]),
                const Text('FEEDS:'),
                const Text('4-6'),
              ],
            ),
          ],
        ),
      ),
    );

    // 4. MENGGABUNGKAN SEMUA KONTEN KIRI (Ke dalam 1 Column)
    final leftColumn = Container(
      padding: const EdgeInsets.fromLTRB(10, 20, 10, 10),

```

```

        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            titleText,
            subTitle,
            ratings,
            iconList,
          ],
        ),
      );

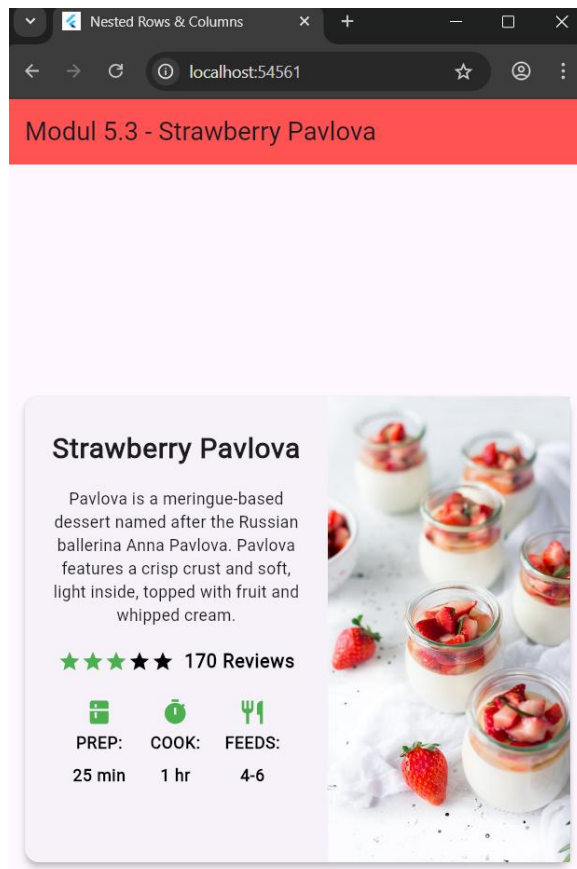
      // 5. GAMBAR UTAMA (Sisi Kanan)
      final mainImage = Image.network(
        'https://images.unsplash.com/photo-1488477181946-6428a0291777?auto=format&fit=crop&w=600&q=80',
        fit: BoxFit.cover,
      );

      // 6. LAYOUT AKHIR (Row Utama pembagi Kiri dan Kanan)
      return Card(
        elevation: 5,
        margin: const EdgeInsets.all(16),
        child: SizedBox(
          height: 400,
          child: Row(
            crossAxisAlignment: CrossAxisAlignment.stretch, // Agar
gambar memenuhi tinggi Card
            children: [
              Expanded(
                flex: 5,
                child: SingleChildScrollView(child: leftColumn),
              ),
              Expanded(
                flex: 4,
                child: mainImage,
              ),
            ],
          ),
        ),
      );
    }

```

```
}
```

Screenshot:



Penjelasan:

Konsep *Nested* (Bersarang) adalah menumpuk Row dan Column di dalam satu sama lain untuk mengatur elemen secara vertikal dan horizontal.

- **Lapis 1 (Pembagi Layar):** Menggunakan Row utama. Row ini membelah layar menjadi 2 bagian: Kiri diisi oleh Column (leftColumn) dan Kanan diisi oleh gambar.
- **Lapis 2 (Isi Kiri):** leftColumn menggunakan Column untuk menyusun teks judul, deskripsi, baris rating, dan baris ikon dari atas ke bawah.
- **Lapis 3 (Baris Rating):** Menggunakan Row untuk menyejajarkan teks ulasan dengan kumpulan bintang. Bintang-bintang itu sendiri dibungkus lagi dalam Row agar sejajar.
- **Lapis 4 (Baris Ikon Info):** Menggunakan Row besar untuk menyejajarkan 3 kategori (PREP, COOK, FEEDS). Di dalam Row ini, masing-masing kategori menggunakan Column karena Ikon, Label, dan Waktu disusun menumpuk ke bawah.

2.4. CustomScrollView

CustomScrollView Widget yang memberikan fleksibilitas tinggi dalam menciptakan efek scrolling khusus. CustomScrollView bekerja dengan komponen yang disebut "slivers" (seperti SliverAppBar, SliverList, dan SliverGrid) untuk menggabungkan berbagai jenis konten scroll dalam satu tampilan tunggal yang mulus.

Source code:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

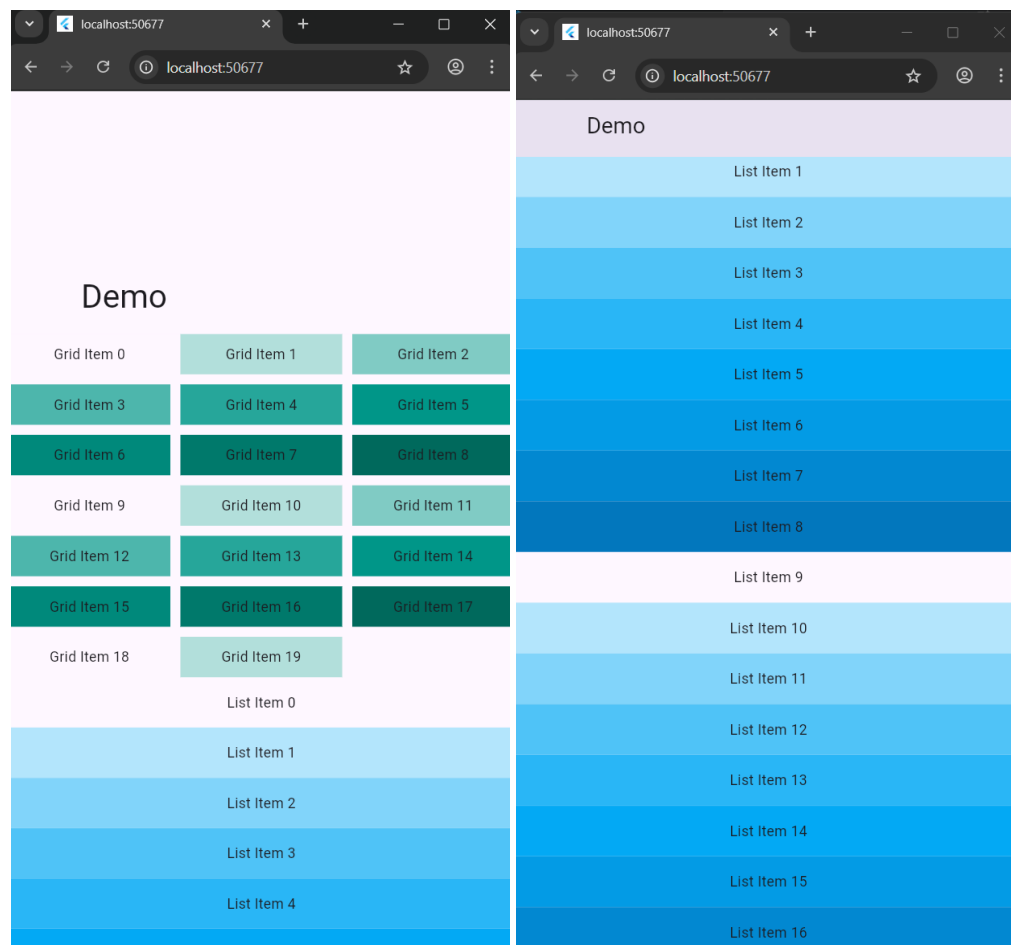
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        body: CustomScrollView(
          slivers: <Widget>[
            const SliverAppBar(
              pinned: true,
              expandedHeight: 250.0,
              flexibleSpace: FlexibleSpaceBar(
                title: Text('Demo'),
              ),
            ),
            SliverGrid(
              gridDelegate: const
SliverGridDelegateWithMaxCrossAxisExtent(
                maxCrossAxisExtent: 200.0,
                mainAxisSpacing: 10.0,
                crossAxisSpacing: 10.0,
                childAspectRatio: 4.0,
              ),
              delegate: SliverChildBuilderDelegate(
                (BuildContext context, int index) {
```

```

        return Container(
            alignment: Alignment.center,
            color: Colors.teal[100 * (index % 9)],
            child: Text('Grid Item $index'),
        );
    },
    childCount: 20,
),
),
SliverFixedExtentList(
    itemExtent: 50.0,
    delegate: SliverChildBuilderDelegate(
        (BuildContext context, int index) {
            return Container(
                alignment: Alignment.center,
                color: Colors.lightBlue[100 * (index % 9)],
                child: Text('List Item $index'),
            );
        },
    ),
),
],
),
),
);
}
}

```

Screenshot:



Penjelasan:

- **CustomScrollView:** Digunakan Ketika ingin menggabungkan beberapa efek scroll yang berbeda di layar yang sama.
- **SliverAppBar:** App bar yang bisa membesar dan mengecil saat layar di-scroll. Properti pinned: true memastikan app bar tetap terlihat saat di-scroll.
- **SliverGrid & SliverFixedExtentList:** Ini adalah versi GridView dan ListView khusus yang dirancang untuk bekerja secara harmonis di dalam CustomScrollView. Pada kode di atas ini, grid dan list digabung dalam satu layar utuh yang bisa di-scroll bersamaan.

UNGUIDED

1. Buat 1 project Flutter yang menampilkan beberapa widget UI berikut:

Yang harus ada:

- a. **Container** → kotak berwarna
- b. **GridView** → minimal 6 item (grid)
- c. **ListView** → 3 item (A, B, C)
- d. **ListView.builder** → list dari data array
- e. **ListView.separated** → list + garis pembatas
- f. **Stack** → tampilan bertumpuk (kotak / text)

Source Code:

main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(
  debugShowCheckedModeBanner: false,
  home: SimplePOSDashboard(),
));

class SimplePOSDashboard extends StatelessWidget {
  const SimplePOSDashboard({super.key});

  @override
  Widget build(BuildContext context) {
    final List<String> keranjang = ['Nasi Goreng', 'Es Teh', 'Kerupuk'];
    final List<String> riwayat = ['TRX-01', 'TRX-02', 'TRX-03'];

    return Scaffold(
      appBar: AppBar(title: const Text('SmartPOS Simple')),
      body: SingleChildScrollView(
        child: Column(
          children: [
            // 1. STACK
            Stack(
              alignment: Alignment.center,
              children: [
                Container(height: 100, width: double.infinity,
                  color: Colors.blue),
                const Text('TOTAL: Rp 150.000', style:
                  TextStyle(fontSize: 24, color: Colors.white)),
              ],
            ),

            // 2. CONTAINER
            Container(
```

```

        margin: const EdgeInsets.all(8),
        padding: const EdgeInsets.all(16),
        color: Colors.greenAccent,
        width: double.infinity,
        child: const Text('Status Printer: Aktif', textAlign:
TextAlign.center),
    ),

    const Divider(),
    const Text('Kategori'),

    // 3. GRIDVIEW
    GridView.count(
      shrinkWrap: true,
      physics: const NeverScrollableScrollPhysics(),
      crossAxisCount: 3,
      padding: const EdgeInsets.all(8),
      children: List.generate(6, (index) => Card(
        color: Colors.orange[200],
        child: Center(child: Text('Menu ${index + 1}')),
      )),
    ),

    const Divider(),
    const Text('Top Menu'),

    // 4. LISTVIEW
    ListView(
      shrinkWrap: true,
      physics: const NeverScrollableScrollPhysics(),
      children: const [
        ListTile(title: Text('A. Ayam Bakar')),
        ListTile(title: Text('B. Bebek Goreng')),
        ListTile(title: Text('C. Cumi Saus Tiram')),
      ],
    ),

    const Divider(),
    const Text('Keranjang'),

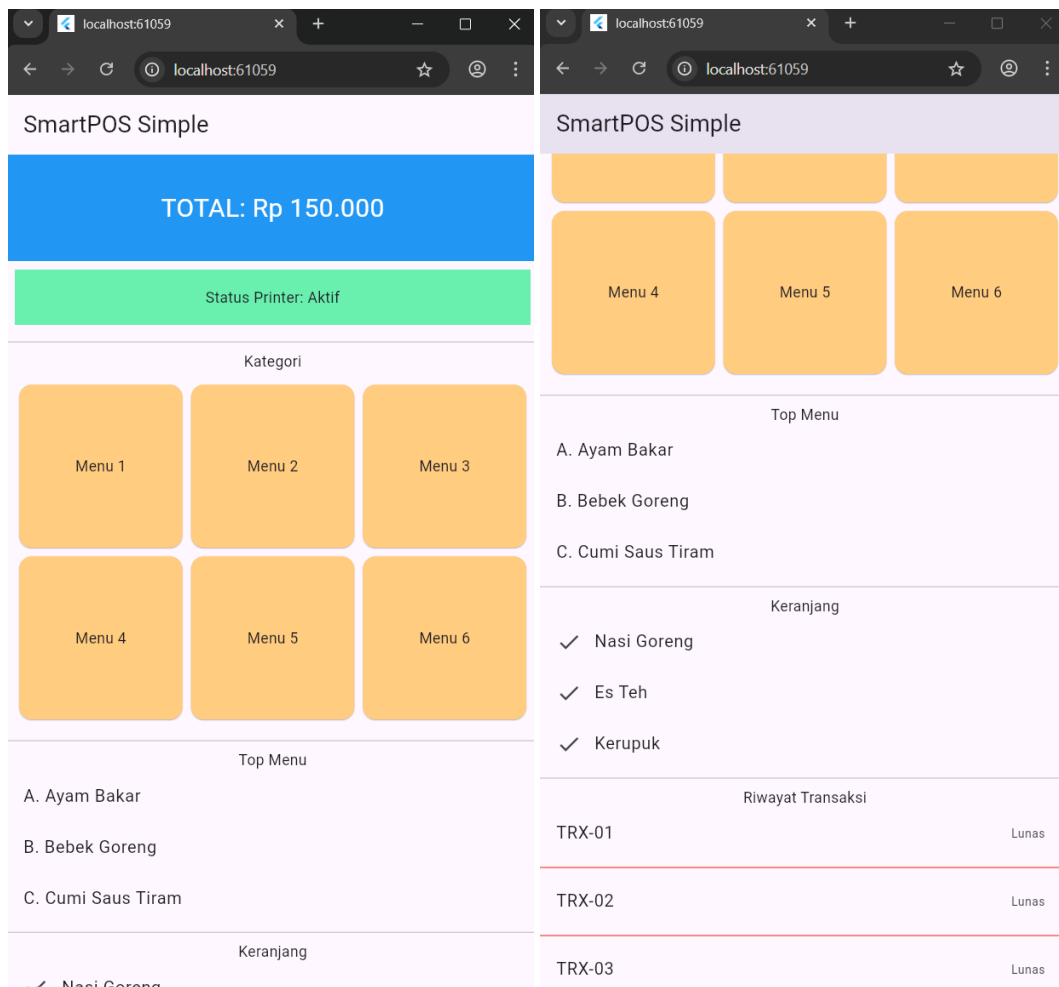
    // 5. LISTVIEW.BUILDER
    ListView.builder(
      shrinkWrap: true,
      physics: const NeverScrollableScrollPhysics(),
      itemCount: keranjang.length,
      itemBuilder: (context, index) => ListTile(
        title: Text(keranjang[index]),
        leading: const Icon(Icons.check),
      ),
    ),

    const Divider(),
    const Text('Riwayat Transaksi'),

```

```
// 6. LISTVIEW.SEPARATED
ListView.separated(
  shrinkWrap: true,
  physics: const NeverScrollableScrollPhysics(),
  itemCount: riwayat.length,
  separatorBuilder: (context, index) => const
Divider(color: Colors.red),
  itemBuilder: (context, index) => ListTile(
    title: Text(riwayat[index]),
    trailing: const Text('Lunas'),
  ),
),
),
],
),
),
);
}
```

Screenshot:



Penjelasan:

Halaman aplikasi ini dibungkus dengan `SingleChildScrollView` dan `Column` agar seluruh isinya bisa di-scroll. Di dalamnya, widget `Stack` digunakan untuk menumpuk teks di atas kotak biru, sementara `Container` berfungsi sebagai kotak hijau sederhana untuk teks status printer.

Untuk menyusun barisan data, kode ini menggunakan empat tata letak utama: `GridView` untuk menampilkan enam kotak menu yang berbaris menyamping, `ListView` untuk membuat tiga baris daftar statis yang ditulis manual, `ListView.builder` untuk mencetak daftar otomatis dari sebuah data array, dan `ListView.separated` yang fungsinya sama seperti builder namun secara otomatis menyisipkan garis batas di antara setiap itemnya. Terakhir, semua widget daftar (`Grid` dan `List`) tersebut wajib menggunakan pengaturan `shrinkWrap: true` dan mematikan efek scroll bawaannya agar tidak berbenturan dan menyebabkan error pada layar.

Link GitHub: <https://github.com/Aplikasi-Berbasis-Platform-S1IF-11-04/Pertemuan-9-Mobile/tree/main/2311102132-Sinta%20Sarwo>